

Pianificazione Robotica e Sistemi Multi-Agente

NECKER

9 Luglio 2026

Indice

1	Fondamenti della Pianificazione Robotica	2
2	Ricerca Euristica e Algoritmo A^*	3
2.1	Ricerca a Sub-Ottimalità Limitata: Algoritmo A_ϵ^* e FOCAL Search	4
3	Assegnazione di Task Dinamici: MAPD	6
3.1	L'algoritmo Token Passing (TP)	7
3.2	Varianti Predittive: Anticipare il Futuro	7
4	Pianificazione Multi-Agente (MAPF e MRPP)	8
4.1	Tipologie di Conflitto e Complessità	8
4.2	L'algoritmo Conflict-Based Search (CBS)	9
4.3	Estensioni: Configurable MAPF (C-MAPF)	10
5	Limiti dei Modelli Fondazionali (LLMs) nella Pianificazione	10
6	Pianificazione del Movimento e Spazio delle Configurazioni	11
7	Sampling-Based Motion Planning (SBMP)	12
7.1	Probabilistic Roadmaps (PRM)	13
7.2	Rapidly-exploring Random Trees (RRT)	13

1 Fondamenti della Pianificazione Robotica

La pianificazione è formalizzata come il problema di selezionare una sequenza di azioni per raggiungere un obiettivo assegnato. Questo dominio rappresenta l'intersezione in cui l'Intelligenza Artificiale converge con la Robotica, sfumando i confini tra robot fisici e agenti software.

- **Pianificazione Deterministica (Open-Loop):** Questo approccio presuppone infatti che il robot abbia una conoscenza perfetta e immutabile dell'ambiente e che non vi siano imprevisti o disturbi casuali durante il movimento. L'obiettivo matematico è calcolare in anticipo una sequenza esatta di azioni di controllo $\mathcal{U} = \{u_1, u_2, \dots, u_n\}$ per guidare il sistema lungo una traiettoria ideale, definita da una serie di stati $x_1, x_2, \dots, x_n$, garantendo che lo stato finale coincida con l'obiettivo ($x_n \in X_g$). Il meccanismo si basa su una funzione di transizione deterministica a tempo discreto:

$$x_{k+1} = f(x_k, u_{k+1}) \quad (1)$$

In parole semplici, se conosciamo lo stato attuale e l'azione che stiamo per compiere, lo stato successivo è già scritto, del tutto prevedibile e privo di incertezze.

In questa fase, lo spazio circostante e gli ostacoli statici noti vengono mappati e discretizzati in strutture geometriche come grafi topologici o griglie di occupazione. La ricerca della rotta ottimale viene poi affidata ad algoritmi classici (come A^* , Dijkstra o pianificatori incrementali), che estraggono la sequenza di passaggi più efficiente sotto vincoli puramente geometrici, ignorando momentaneamente i dettagli fisici e le perturbazioni del movimento reale.

- **Feedback Planning e Controllo Politico (Closed-Loop):** In scenari dinamici, seguire alla cieca una sequenza rigida pianificata ad anello aperto porta quasi sempre al fallimento. Il *Feedback Planning* supera questo limite introducendo il concetto di politica (*policy*) $\pi : X \rightarrow U$. Invece di calcolare una lista fissa di azioni, il robot apprende una regola decisionale che gli dice esattamente cosa fare in tempo reale per *qualunque* situazione o stato $x \in X$ in cui possa trovarsi: $u = \pi(x)$. Questo approccio presuppone che il sistema sia in grado di monitorare costantemente la propria posizione e il proprio orientamento sfruttando i sensori di bordo.

Quando gli effetti delle azioni non sono certi ma probabilistici, il problema viene formalizzato rigorosamente attraverso i **Processi Decisionali di Markov (MDP)**, definiti dalla tupla $\langle X, U, P, R, \gamma \rangle$. In questo caso, l'incertezza del mondo fisico viene catturata dalla probabilità di transizione:

$$P(x_{k+1} \mid x_k, u_k) \quad (2)$$

che definisce quanto sia probabile finire in un determinato stato successivo partendo da quello attuale e applicando una specifica azione. Il comportamento del robot viene guidato da una funzione di ricompensa immediata $R(x_k, u_k)$ (che premia i comportamenti corretti e penalizza gli errori) e da un fattore di sconto $\gamma \in [0, 1)$ che pesa l'importanza dei risultati futuri rispetto a quelli immediati.

2 Ricerca Euristica e Algoritmo A^*

L'algoritmo A^* è una tecnica di ricerca *best-first* informata, ampiamente utilizzata per trovare il cammino di costo minimo tra un nodo di partenza e un nodo obiettivo all'interno di un grafo. A differenza degli algoritmi di ricerca cieca, A^* sfrutta la conoscenza specifica del dominio per orientare l'esplorazione, espandendo i nodi in modo da minimizzare la seguente funzione di costo:

$$f(s) = g(s) + h(s) \quad (3)$$

dove:

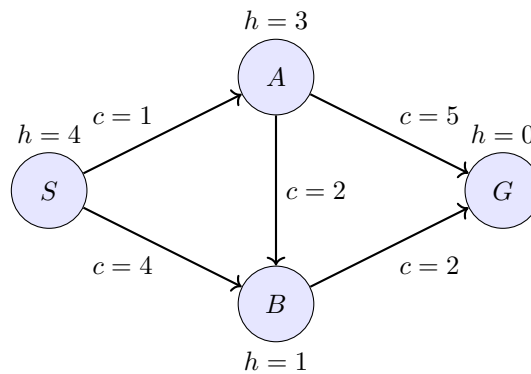
- $g(s)$ è il costo effettivo accumulato per raggiungere lo stato s a partire dal nodo iniziale.
- $h(s)$ è l'**euristica**, ovvero una stima del costo residuo necessario per arrivare all'obiettivo partendo da s .

Funzionamento dell'algoritmo

Il processo di ricerca mantiene generalmente due strutture dati principali: una *Open List* (o frontiera), che contiene i nodi scoperti ma non ancora espansi, e una *Closed List* (EXL), che tiene traccia dei nodi già analizzati.

L'algoritmo procede iterativamente secondo questi passi:

1. Estrae dalla Open List il nodo con il valore $f(s)$ minimo.
2. Se questo nodo corrisponde allo stato obiettivo, l'algoritmo termina e ricostruisce il percorso ottimo a ritroso.
3. Altrimenti, il nodo viene spostato nella Closed List ed "espanso": vengono generati i suoi nodi successori.
4. Per ogni successore, si calcolano i nuovi valori di g , h (dipende dal problema) ed f . Se il successore non è in nessuna lista, viene aggiunto alla Open List. Se è già presente ma il nuovo percorso offre un costo g inferiore, il suo valore viene aggiornato.



- **Inizializzazione:** Si inserisce il nodo di partenza S nella Open List.
 - $g(S) = 0, h(S) = 4 \implies f(S) = 0 + 4 = 4$.
 - **Open List:** $\{S(f=4)\}$
 - **Closed List:** $\{\}$
- **Passo 1: Espansione di S** Si estrae il nodo con f minore dalla Open List, ovvero S . Poiché $S \neq G$, lo si inserisce nella Closed List e si generano i suoi successori, A e B .
 - Per A : $g(A) = g(S) + c(S, A) = 0 + 1 = 1$. Essendo $h(A) = 3$, si ha $f(A) = 1 + 3 = 4$.
 - Per B : $g(B) = g(S) + c(S, B) = 0 + 4 = 4$. Essendo $h(B) = 1$, si ha $f(B) = 4 + 1 = 5$.
 - **Open List:** $\{A(f=4), B(f=5)\}$ (ordinata per f)
 - **Closed List:** $\{S\}$

- **Passo 2: Espansione di A** Si estrae il nodo con f minore dalla Open List, ovvero A . Si sposta A nella Closed List e si valutano i suoi vicini, B e G .
 - Per G : $g(G) = g(A) + c(A, G) = 1 + 5 = 6$. Essendo $h(G) = 0$, si ha $f(G) = 6 + 0 = 6$.
 - Per B : si valuta il percorso passante da A . Il nuovo costo sarebbe $g_{new}(B) = g(A) + c(A, B) = 1 + 2 = 3$. Poiché questo costo (3) è inferiore al costo $g(B)$ precedentemente calcolato (4), il valore di B viene aggiornato nella Open List: $g(B) = 3 \implies f(B) = 3 + 1 = 4$.
 - **Open List:** $\{B(f=4), G(f=6)\}$
 - **Closed List:** $\{S, A\}$
- **Passo 3: Espansione di B** Si estrae il nodo con f minore, ovvero B . Si sposta B nella Closed List e si generano i suoi successori. L'unico successore non ancora in Closed List è G .
 - Per G : si valuta il percorso passante da B . Il nuovo costo sarebbe $g_{new}(G) = g(B) + c(B, G) = 3 + 2 = 5$. Poiché questo costo (5) è inferiore al costo $g(G)$ precedente (6), il valore del nodo obiettivo viene aggiornato nella Open List: $g(G) = 5 \implies f(G) = 5 + 0 = 5$. (se fosse costato di più, semplicemente non si aggiornava nella lista open list e si sarebbe preso il val precedente)
 - **Open List:** $\{G(f=5)\}$
 - **Closed List:** $\{S, A, B\}$
- **Passo 4: Estrazione di G (Terminazione)** Si estrae dalla Open List il nodo con f minore, che in questo caso è proprio l'obiettivo G .
 - Poiché il nodo estratto è il *goal*, l'algoritmo termina con successo.
 - Il cammino ottimo viene ricostruito a ritroso risalendo i puntatori dei genitori: $S \rightarrow A \rightarrow B \rightarrow G$, con un costo totale effettivo pari a $g(G) = 5$.

2.1 Ricerca a Sub-Ottimalità Limitata: Algoritmo A_ϵ^* e FOCAL Search

Nelle applicazioni reali, la ricerca del cammino perfettamente ottimo tramite A^* classico può richiedere un dispendio insostenibile di tempo e memoria, poiché l'algoritmo è costretto a esplorare tutti i nodi che potrebbero teoricamente offrire un costo inferiore. Per ovviare a questo limite, si introduce il concetto di **ricerca a sub-ottimalità limitata** (*bounded suboptimal search*).

L'obiettivo non è più trovare il cammino minimo assoluto C^* , ma garantire la generazione di una soluzione il cui costo totale C sia matematicamente limitato da un fattore di tolleranza $\epsilon \geq 0$ scelto dall'utente:

$$C \leq (1 + \epsilon) \cdot C^* \quad (4)$$

Il capostipite di questo approccio è l'algoritmo A_ϵ^* (o *Focal Search*), il quale rilassa il vincolo di espansione rigorosa dei nodi ottimali (*OPT*) per velocizzare drasticamente l'esplorazione.

La struttura: OPEN List e FOCAL List

Mentre A^* classico seleziona i nodi basandosi esclusivamente sul valore minimo della funzione $f(s) = g(s) + h(s)$, la *Focal Search* sdoppia la gestione della frontiera introducendo una seconda lista dinamica:

1. **OPEN List:** Mantiene la medesima funzione del caso classico, contenendo tutti i nodi generati e non ancora espansi, ordinati per valori crescenti di $f(s)$. Sia f_{min} il valore di f più basso attualmente presente in questa lista:

$$f_{min} = \min_{s \in \text{OPEN}} f(s) \quad (5)$$

Il valore f_{min} rappresenta il limite inferiore (*lower bound*) corrente sul costo della soluzione ottima.

2. **FOCAL List:** È un sottoinsieme mobile della *OPEN List*. Essa contiene solo i nodi il cui costo stimato $f(s)$ non supera f_{min} di oltre il fattore di tolleranza stabilito:

$$\text{FOCAL} = \{s \in \text{OPEN} \mid f(s) \leq (1 + \epsilon) \cdot f_{min}\} \quad (6)$$

Strategia di Espansione e Relazione con *OPT*

I nodi appartenenti all'insieme dei cammini ottimali (*OPT*) sono implicitamente protetti dal limite inferiore f_{min} . Finché un nodo si trova all'interno della *FOCAL List*, l'algoritmo ha la certezza matematica che, espandendolo, non violerà il vincolo di sub-ottimalità bound $(1 + \epsilon)$.

La vera potenza dell'algoritmo risiede nel criterio di selezione all'interno della *FOCAL*: Anziché estrarre il nodo con il valore $f(s)$ minore, l'algoritmo ordina ed estrae dalla *FOCAL* il nodo che minimizza una **seconda funzione euristica indipendente**, spesso indicata con $h_F(s)$ o $d(s)$.

La funzione $d(s)$ stima tipicamente il **numero residuo di passi (archi)** mancanti per raggiungere l'obiettivo, ignorando completamente il loro costo monetario o geometrico.

Vantaggi dell'Approccio

In questo modo, l'algoritmo si comporta in modo estremamente efficiente:

- **Velocità:** Scegliendo il nodo più vicino in termini di passaggi all'obiettivo (tramite $d(s)$), l'algoritmo "punta" dritto al *goal* quasi come una ricerca *greedy*, riducendo il numero di nodi espansi anche di diversi ordini di grandezza.
- **Garanzia formale:** La *OPEN List* e il vincolo imposto sulla *FOCAL* fungono da barriera di sicurezza. Se l'euristica secondaria dovesse spingere l'algoritmo verso nodi troppo costosi, questi supererebbero la soglia $(1 + \epsilon) \cdot f_{min}$ e verrebbero automaticamente estromessi dalla *FOCAL*, costringendo l'algoritmo a ritornare verso i nodi vicini a quelli ottimali (*OPT*).

Proprietà dell'Euristica

Per garantire l'ottimalità della soluzione trovata, l'euristica $h(s)$ deve rispettare precise proprietà matematiche:

- **Ammissibilità:** Un'euristica è ammissibile se non sovrastima mai il costo del cammino minimo verso l'obiettivo. In termini pratici, deve essere rigorosamente ottimistica: per ogni nodo s , $h(s) \leq h^*(s)$, dove $h^*(s)$ è il costo reale residuo.

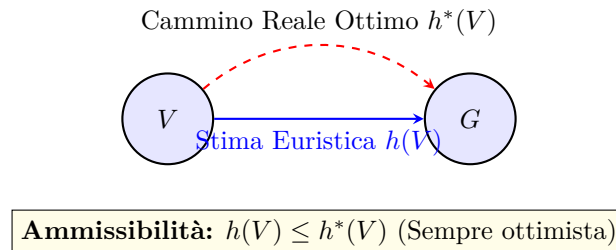


Figura 1: Proprietà di Ammissibilità: L'euristica $h(V)$ sottostima sempre il costo reale $h^*(V)$ necessario per raggiungere l'obiettivo G .

- **Consistenza (o Monotonia):** Si tratta di un requisito più stringente, fondamentale per le ricerche che utilizzano liste chiuse (EXL). Dati due stati V e U connessi da un'azione a , l'euristica h è consistente se rispetta la disuguaglianza triangolare:

$$h(V) \leq c(V, a, U) + h(U) \quad (7)$$

dove $c(V, a, U)$ è il costo esatto per passare da V a U . Un'euristica consistente è sempre implicitamente ammissibile. Se si utilizza la lista chiusa (EXL) con un'euristica che è *solo* ammissibile ma non consistente, l'algoritmo non garantisce l'ottimalità, a meno di non re-inserire nella Open List nodi già esplorati se viene trovato un percorso migliore per raggiungerli (operazione molto costosa).

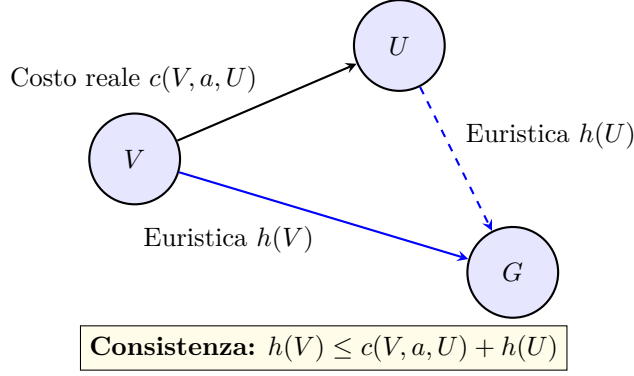


Figura 2: Proprietà di Consistenza: L'euristica deve rispettare la disuguaglianza triangolare tra due nodi vicini e il Goal.

3 Assegnazione di Task Dinamici: MAPD

Il problema del **Multi-Agent Pickup and Delivery (MAPD)** modella scenari reali in cui una flotta di agenti (ad esempio, robot in un magazzino logistico) deve gestire un flusso continuo di richieste. A differenza della pianificazione classica in cui tutti i task sono noti a priori, nel MAPD i task τ_i arrivano dinamicamente a *runtime*.

Ogni task τ_i è caratterizzato da due parametri fondamentali:

- s_i : il vertice di *pickup* (luogo di prelievo).
- g_i : il vertice di *delivery* (luogo di consegna).

Un task si considera completato con successo quando un agente assegnato viaggia dalla sua posizione attuale fino a s_i , preleva l'oggetto, e successivamente naviga verso g_i per consegnarlo, il tutto senza collidere con altri agenti.

Le prestazioni del sistema MAPD vengono valutate principalmente attraverso due metriche:

1. **Makespan:** Il numero totale di *step* temporali necessari affinché l'ultimo task dell'intero sistema venga completato.
2. **Service time:** Il tempo medio di attesa di un task, misurato dall'istante in cui appare nel sistema fino al momento in cui viene effettivamente assegnato e prelevato.

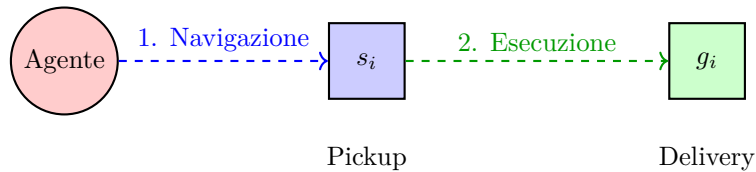


Figura 3: Ciclo di vita base di un task nel problema MAPD: l'agente libero viaggia verso il nodo di pickup per poi dirigersi al nodo di delivery.

3.1 L'algoritmo Token Passing (TP)

Per risolvere il MAPD in ambienti altamente dinamici, un approccio standard è l'algoritmo **Token Passing (TP)**. Si tratta di un algoritmo *online* e *decentralizzato*.

Il cuore del sistema è un "Token" condiviso, ovvero una struttura dati che contiene i percorsi attualmente pianificati da tutti gli agenti e la lista dei task non ancora assegnati. Quando un agente diventa libero (ha completato il suo task precedente), richiede il Token. Una volta ottenuto, l'agente sceglie un nuovo task dalla lista, pianifica un percorso non collidente (rispettando i percorsi futuri già prenotati dagli altri agenti nel Token) e infine passa il Token all'agente successivo (il token è il permesso di scrittura, questo serve per impedire che due agenti scrivano allo stesso tempo 2 percorsi che collidono pensando di non collidere con nessuno).

Se **non ci sono task eleggibili** (la coda è vuota), un agente libero non rimane fermo dove si trova, poiché rischierebbe di bloccare i percorsi altrui. L'algoritmo TP impone che l'agente si diriga verso una *safe location* (luogo sicuro, come un parcheggio o un punto di ricarica) pianificando un percorso non collidente e mettendosi in attesa.

3.2 Varianti Predittive: Anticipare il Futuro

L'algoritmo TP base reagisce solo ai task già esistenti. Tuttavia, in molti scenari reali, l'arrivo dei task segue modelli probabilistici prevedibili. Per abbattere il *Service time*, sono state sviluppate varianti predittive che permettono agli agenti di spostarsi strategicamente prima ancora che un task venga generato.

- **TP-m2 (Speculazione base):** Questa variante introduce il concetto di *speculazione*. Se non ci sono task attuali, anziché andare in una *safe location* generica, l'agente calcola un punteggio per vari vertici "candidati" in cui è probabile che appaia un task a breve. Il punteggio S per un candidato s è calcolato come:

$$S = \frac{\sum P(t, s)}{1 + h(v, s)} \quad (8)$$

dove il numeratore rappresenta la probabilità aggregata P che appaia un task nel vertice s all'istante di tempo t , mentre $h(v, s)$ è la distanza euristica (il costo del viaggio) dalla posizione attuale v dell'agente al candidato s . L'agente si dirigerà verso il vertice candidato che massimizza S , bilanciando così l'alta probabilità con la vicinanza spaziale.

- **TP-m1 (Priorità speculativa):** Si spinge oltre rispetto a TP-m2. In TP-m1, un agente valuta se sia più conveniente prendere in carico un task *attualmente esistente* ma molto lontano, oppure ignorarlo e speculare su un task *futuro* che ha un'altissima probabilità di apparire molto vicino a lui. Questo approccio riduce drasticamente i tempi morti, ma introduce il rischio di *starvation* (task lontani e isolati che non vengono mai serviti). Per mitigare questo problema, il sistema può forzare l'assegnazione bloccando temporaneamente la generazione di nuovi task finché quelli vecchi non vengono smaltiti.
- **Preemption (Prelazione):** Per rendere la speculazione ancora più efficace, si introduce la logica di prelazione. Quando un agente decide di speculare verso un vertice candidato, guadagna dei "diritti di prelazione" su un'area geografica (una *zona di preemption* di raggio d) circostante il vertice obiettivo. Se un nuovo task appare in questa zona mentre l'agente è in viaggio, l'agente se lo auto-assegna istantaneamente, interrompendo il viaggio speculativo per iniziare immediatamente l'esecuzione del task, risparmiando tempo prezioso di ri-assegnazione e senza dover richiedere il token (scrive direttamente in memoria la sua traiettoria dato che è vicinissimo all'obiettivo ed è difficile che un percorso di un altro collida).

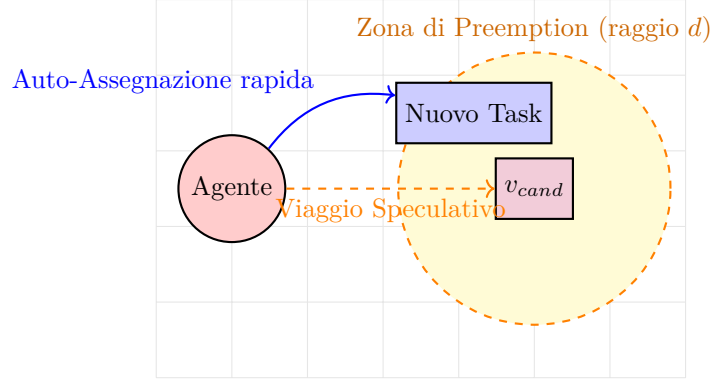


Figura 4: Logica di Preemption: l'agente sta viaggiando verso un nodo candidato v_{cand} . Appena compare un nuovo task all'interno della sua zona di preemption (raggio d), l'agente devia e se lo auto-assegna immediatamente.

4 Pianificazione Multi-Agente (MAPF e MRPP)

Il **Multi-Robot Path Planning (MRPP)** è il problema che richiede la determinazione di percorsi sicuri ed efficienti per completare task distribuiti su più unità robotiche, tenendo spesso conto di vincoli fisici e cinematici. La sua generalizzazione formale e algoritmica prende il nome di **Multi-Agent Path Finding (MAPF)**.

Nel MAPF astratto, l'obiettivo è trovare percorsi ottimali e privi di collisioni (*non interferenti*) per un insieme di k agenti. Il problema è tipicamente modellato su un grafo $G = (V, E)$, in cui ogni agente $i \in \{1, \dots, k\}$ possiede un vertice di partenza s_i e un vertice obiettivo g_i . Il tempo è discretizzato e a ogni passo temporale un agente può spostarsi in un vertice adiacente o attendere nel vertice attuale.

4.1 Tipologie di Conflitto e Complessità

Per garantire che i percorsi siano sicuri, una soluzione MAPF non deve presentare conflitti. I due conflitti fondamentali sono:

- **Conflitto di vertice (*Vertex Conflict*):** Due agenti tentano di occupare lo stesso vertice v nello stesso istante di tempo t .
- **Conflitto di arco (*Edge Conflict*):** Due agenti tentano di attraversare lo stesso arco (u, v) in direzioni opposte nel medesimo istante di tempo, "scambiandosi" di posto.

Il problema richiede generalmente la minimizzazione di una funzione di costo globale, come la somma delle durate dei singoli percorsi (*Sum of Costs*) o il tempo di arrivo dell'ultimo agente (*Makespan*). Trovare una soluzione ottimale per il MAPF è un problema classificato come NP-Hard, poiché lo spazio degli stati cresce esponenzialmente al crescere del numero di agenti.

Vertex Conflict (al tempo t)



Edge Conflict (al tempo t)

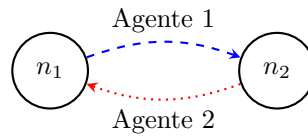


Figura 5: Rappresentazione dei due vincoli fondamentali nel MAPF. A sinistra, collisione su un vertice condiviso. A destra, collisione frontale su un arco.

4.2 L'algoritmo Conflict-Based Search (CBS)

Attualmente, lo stato dell'arte per le soluzioni esatte del MAPF è il **Conflict-Based Search (CBS)**. CBS è un algoritmo completo e ottimale che opera su un'architettura gerarchica a due livelli, disaccoppiando la ricerca globale dalla pianificazione del singolo agente.

- **Low level (Ricerca del singolo agente):** A questo livello, l'algoritmo calcola il cammino ottimale per un *singolo* agente dal suo nodo di partenza al suo traguardo, rispettando una lista di vincoli spazio-temporali che gli sono stati imposti. Questo viene solitamente risolto con una variante spaziotemporale di A^* .
- **High level (Albero dei Vincoli):** Il livello alto esegue una ricerca *best-first* su un Albero dei Vincoli (*Constraint Tree* - CT). Ogni nodo del CT contiene:
 1. Un set di vincoli (interdizioni spazio-temporali del tipo: l'agente i non può stare in v al tempo t).
 2. Una soluzione (un insieme di percorsi per tutti gli agenti che rispetta i vincoli del nodo).
 3. Il costo totale della soluzione.

Esempio:

- **Nodo Radice del CT (Nessun vincolo iniziale):** Il livello alto parte con un insieme di vincoli vuoto. Chiede al livello basso (Spazio-Temporale A^*) di calcolare i percorsi individuali più brevi per ciascun agente, ignorando gli altri.
 - **Low-level:** A_1 pianifica il percorso $[S_1 \rightarrow B \rightarrow C \rightarrow G_1]$. Si trova in C al tempo $t = 2$.
 - **Low-level:** A_2 pianifica il percorso $[S_2 \rightarrow D \rightarrow C \rightarrow G_2]$. Si trova in C al tempo $t = 2$.
 - **High-level (Verifica):** Il livello alto esamina i percorsi e trova un **conflitto**: $\langle A_1, A_2, C, t = 2 \rangle$. Entrambi gli agenti collidono nella casella C al tempo 2.

Il livello alto decide quindi di **aggiungere due rami** all'albero (due nodi figli), provando a risolvere il conflitto in due modi opposti.

-
- **Nodo Figlio Sinistro (Vietato ad A_1):** Il livello alto impone il vincolo: "*L'agente A_1 non può stare in C al tempo $t = 2$* ". La limitazione non si applica ad A_2 .
 - **Low-level:** Il livello basso ricalcola il percorso **solo per** A_1 , poiché per A_2 non è cambiato nulla. Il vecchio percorso di A_1 non è più valido perché viola il vincolo.
 - **Nuovo piano per A_1 :** A^* spazio-temporale trova una deviazione o fa aspettare A_1 di un passo. Il nuovo percorso è $[S_1 \rightarrow B \rightarrow B \rightarrow C \rightarrow G_1]$ (costo aumentato di 1, passa da C al tempo $t = 3$).
 - **High-level (Verifica):** Il livello alto riconrolla tutti i percorsi complessivi. Adesso A_2 passa da C al tempo 2, mentre A_1 ci passa al tempo 3. **Nessun conflitto rimasto.** Questo nodo diventa una soluzione valida ed ottimale.
-
- **Nodo Figlio Destro (Vietato ad A_2):** In parallelo (essendo una ricerca *best-first*), il livello alto esplora anche l'alternativa: "*L'agente A_2 non può stare in C al tempo $t = 2$* ".
 - **Low-level:** Questa volta il percorso di A_1 resta quello originario. Viene ricalcolato **solo il percorso di A_2** forzandolo a evitare C al tempo 2.
 - **Nuovo piano per A_2 :** A_2 trova una strada alternativa girando attorno a C .
 - **High-level (Verifica):** Anche questo ramo viene verificato. Se non ci sono conflitti, il livello alto ne confronterà il costo con quello del ramo sinistro e sceglierà il percorso globale che minimizza la somma dei costi di tutti gli agenti.

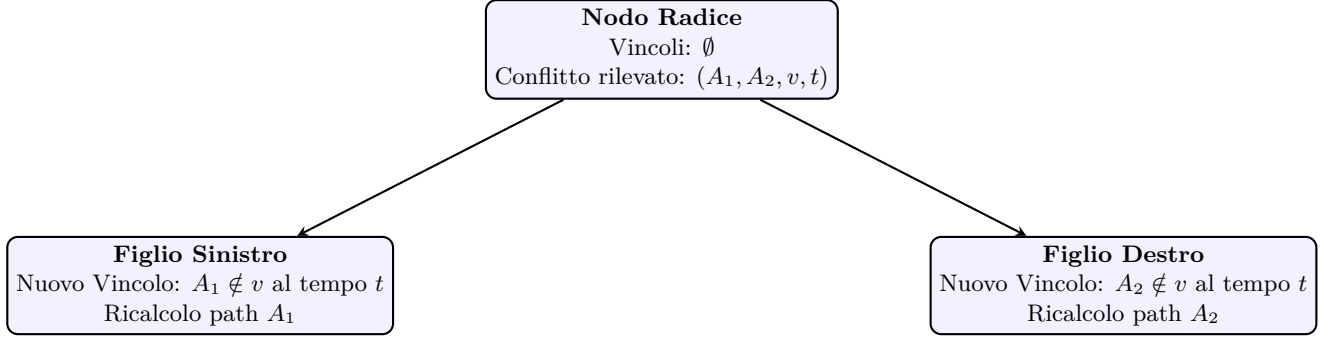


Figura 6: Processo di *split* nell'albero dei vincoli (High Level) di CBS. La risoluzione di un conflitto tra due agenti A_1 e A_2 genera due scenari paralleli esplorabili.

4.3 Estensioni: Configurable MAPF (C-MAPF)

Un'evoluzione naturale legata ai sistemi logistici moderni (come i magazzini automatizzati di Amazon Robotics) è il **C-MAPF (Configurable MAPF)**. Mentre nel MAPF tradizionale l'ambiente è statico, nel C-MAPF si introduce un'ulteriore complessità dimensionale permettendo agli agenti di modificare dinamicamente il grafo stesso (ad esempio, trasportando scaffali o ostacoli da un vertice all'altro). Questa riconfigurabilità, sebbene aumenti enormemente il *branching factor* del problema, permette l'ottimizzazione congiunta sia dei percorsi dei robot sia della planimetria temporanea dell'ambiente di lavoro.

5 Limiti dei Modelli Fondazionali (LLMs) nella Pianificazione

Negli ultimi anni, l'avvento dei *Large Language Models* (LLMs) ha sollevato interrogativi sulla loro reale capacità di agire come agenti pianificatori. Sebbene questi modelli mostrino pattern emergenti complessi e una notevole fluidità linguistica grazie ai meccanismi di *Attention*, falliscono sistematicamente nella pianificazione deterministica reale.

Come sottolineato dal ricercatore Subbarao Kambhampati, l'errore fondamentale risiede nell'antropomorfizzare i risultati del modello: non bisogna scambiare la generazione sequenziale di token (una predizione statistica) per vere "tracce di ragionamento" logico. Gli LLMs funzionano come approssimatori di funzioni auto-regressivi: possono "mimare" la distribuzione statistica di una traccia prodotta da un algoritmo formale (come A^*) se ne hanno viste abbastanza nei dati di addestramento, ma **non possiedono una rappresentazione causale interna** del mondo. La pianificazione richiede esplorazione combinatoria, validazione degli stati, *lookahead* (previsione delle conseguenze) e, soprattutto, meccanismi di *backtracking* formale per riprendersi dagli errori (come fa la *Open List* di A^*). Poiché gli LLM generano la soluzione in un singolo passaggio predittivo (sprovvisti di un risolutore di grafi integrato), spesso producono piani che sembrano linguisticamente plausibili ma che, all'atto pratico, violano le leggi fisiche o le precondizioni logiche del problema (allucinazioni di pianificazione).

6 Pianificazione del Movimento e Spazio delle Configurazioni

Per poter utilizzare algoritmi di ricerca nello spazio fisico, è necessario mappare il **Workspace** $\mathcal{W}$ (il mondo reale in cui vive il robot, tipicamente in $\mathbb{R}^2$ o $\mathbb{R}^3$) nel **Configuration Space (C-Space)** $\mathcal{C}$.

Nel $\mathcal{C}$ -Space, il robot perde la sua forma geometrica e viene collassato in un singolo **punto** matematico. Le coordinate di questo punto descrivono univocamente tutti i *Gradi di Libertà* (DOF - *Degrees of Freedom*) del robot. Ad esempio, un robot differenziale piatto in $\mathcal{W} \in \mathbb{R}^2$ ha 3 DOF: la posizione (x, y) e l'orientamento (θ) . Pertanto, il suo $\mathcal{C}$ -Space sarà tridimensionale.

Ostacoli e Vincoli nel $\mathcal{C}$ -Space

Per compensare il fatto che il robot sia stato ridotto a un punto, gli ostacoli del mondo reale vengono geometricamente "gonfiati" (tramite una operazione nota come Somma di Minkowski) in base alla forma e al raggio del robot. Il $\mathcal{C}$ -Space si divide così in due insiemi disgiunti:

- **Spazio degli Ostacoli** $\mathcal{O}$: Tutte le configurazioni in cui il robot colliderebbe con l'ambiente o con se stesso.
- **Spazio Libero** $\mathcal{C}_{free} = \mathcal{C} \setminus \mathcal{O}$: L'insieme di tutte le configurazioni sicure.

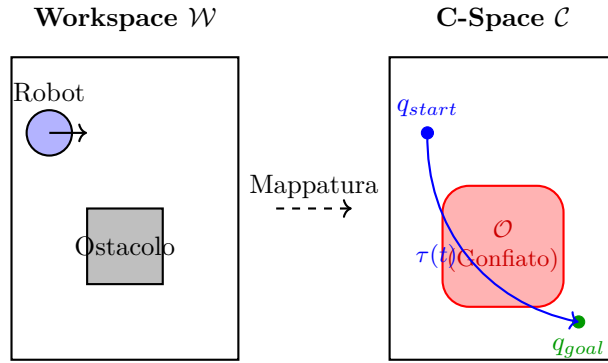


Figura 7: Dal Workspace al C-Space: per trattare il robot come un punto matematico q (facilitando la ricerca del cammino τ), gli ostacoli vengono espansi del raggio del robot per formare la regione di collisione $\mathcal{O}$.

Il problema formale della pianificazione diventa quindi la ricerca di una funzione continua $\tau : [0, 1] \rightarrow \mathcal{C}_{free}$, tale che $\tau(0) = q_{start}$ e $\tau(1) = q_{goal}$.

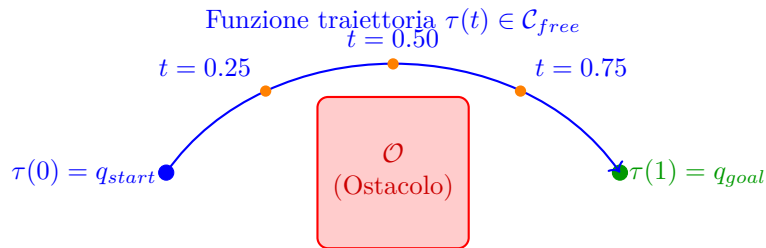


Figura 8: Rappresentazione geometrica della funzione $\tau : [0, 1] \rightarrow \mathcal{C}_{free}$. L'avanzare del parametro t da 0 a 1 "disegna" il percorso continuo dallo start al goal, aggirando le zone di collisione.

Durante questa ricerca, il sistema è soggetto a due tipologie di **vincoli cinematici**:

1. **Vincoli Olonomi**: Equazioni del tipo $g(q) = 0$ che limitano le posizioni raggiungibili, inoltre ogni vincolo olonomo indipendente riduce i gradi di libertà del sistema di 1. Si consideri un vagone sulle montagne russe. Pur vivendo in uno spazio tridimensionale ($\mathbb{R}^3$), il vagone è vincolato a scorrere lungo una guida fisica fissa (la rotaia). Le equazioni che descrivono la rotaia sono del tipo $g(q) = 0$. A causa di questo vincolo, la posizione e l'orientamento del vagone sono determinati da un'unica variabile: la distanza percorsa lungo il binario. Il sistema ha perso i suoi Gradi di Libertà originari, riducendosi a 1 solo DOF. Il robot *non può* raggiungere punti al di fuori della rotaia.

2. **Vincoli Non-Olonomi:** Equazioni differenziali del tipo $u(q)\dot{q} = 0$ che limitano le *velocità* e le direzioni del moto, senza ridurre i DOF. L'esempio classico è un'automobile: può raggiungere qualsiasi coordinata (x, y) nel parcheggio (3 DOF preservati), ma non può muoversi lateralmente (vincolo sulla derivata). Per capire meglio, la componente laterale della velocità deve essere zero ($v_y = 0$ nel sistema di riferimento locale). Di conseguenza, per spostarsi lateralmente, l'auto non può viaggiare in linea retta, ma deve eseguire una complessa manovra (come il parcheggio a "S").

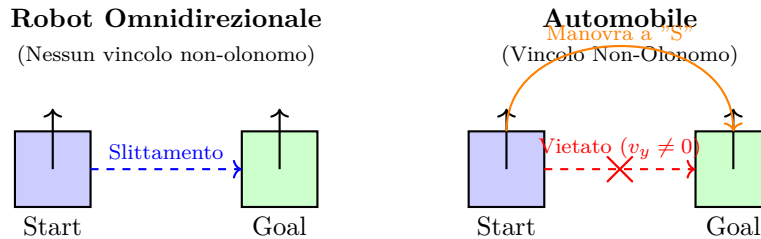


Figura 9: Differenza pratica nella navigazione. Un robot omnidirezionale può scorrere lateralmente. Un'auto (soggetta a vincoli non-olonomi sulle ruote) non può slittare lateralmente, costringendo il *Local Planner* a calcolare traiettorie curve più complesse per raggiungere lo stesso obiettivo.

Local Planners per l'esecuzione della Traiettoria

Per navigare rispettando la cinematica e i vincoli non-olonomi, e per reagire ad ostacoli dinamici non previsti, si utilizzano algoritmi di *Local Planning* che operano in tempo reale ad alta frequenza:

- **Dynamic Window Approach (DWA):** Invece di calcolare un percorso geometrico, DWA lavora nello *spazio delle velocità* (traslazionale v e rotazionale ω). Ad ogni istante, campiona una "finestra" di velocità raggiungibili dal robot nel prossimo *time-step* (date le sue accelerazioni massime). Per ogni coppia (v, ω) , simula una breve traiettoria circolare e la valuta tramite una funzione obiettivo che premia l'avanzamento verso il goal e penalizza la vicinanza agli ostacoli. È computazionalmente leggero e perfetto per robot differenziali su griglie, ma tende a rimanere bloccato nei minimi locali in ambienti molto ristretti (es. corridoi a U).
- **Timed Elastic Bands (TEB):** Formulato come un problema di ottimizzazione non-lineare multi-obiettivo nel dominio spazio-temporale. L'algoritmo inizializza una traiettoria grezza tra start e goal. Questa traiettoria viene modellata come una serie di punti uniti da "molle elastiche". Forze virtuali attrattive tirano la traiettoria verso il traguardo e ne mantengono la fluidità (rispettando la cinematica), mentre forze repulsive (generate dagli ostacoli) "deformano" l'elastico per mantenere la distanza di sicurezza. Modificando i pesi temporali, TEB garantisce che il robot raggiunga il goal nel minor tempo possibile senza violare limiti di velocità o accelerazione.

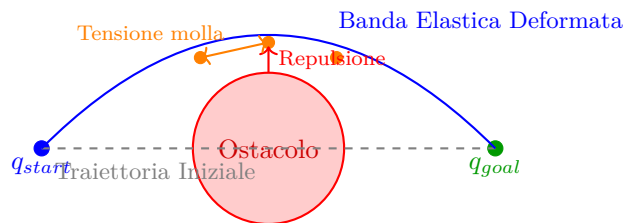


Figura 10: Concetto del Local Planner TEB (*Timed Elastic Band*): La traiettoria è modellata come un elastico che viene deformato dalle forze repulsive dell'ostacolo, mantenendo la coerenza cinematica tramite la tensione interna.

7 Sampling-Based Motion Planning (SBMP)

Quando un robot possiede molti gradi di libertà (ad esempio un braccio robotico con 6 o 7 giunti), il suo *C-Space* diventa uno spazio a 6 o 7 dimensioni. In spazi ad alta dimensionalità, costruire o discretizzare esplicitamente

l'intero spazio per mappare gli ostacoli (creando una *Roadmap* esatta) diventa computazionalmente intrattabile a causa della "maledizione della dimensionalità".

Le tecniche di **Sampling-Based Motion Planning (SBMP)** aggirano questo problema rinunciando a mappare tutto lo spazio. Al contrario, esplorano $\mathcal{C}_{free}$ "alla cieca" tramite un campionamento probabilistico: generano configurazioni casuali e verificano se sono valide (non in collisione). Sebbene perdano la garanzia assoluta di trovare il percorso ottimo (*ottimalità*), offrono una proprietà molto potente: la **Completezza Probabilistica**. Se esiste una soluzione, la probabilità che l'algoritmo la trovi tende a 1 all'aumentare del tempo di esecuzione.

I due algoritmi fondamentali di questa famiglia sono PRM e RRT.

7.1 Probabilistic Roadmaps (PRM)

PRM è un algoritmo **multi-query**, progettato per ambienti statici in cui la mappa non cambia e in cui verranno richieste molteplici pianificazioni nel tempo. Si divide in due fasi distinte:

1. **Fase di Learning (Costruzione offline):** L'algoritmo campiona casualmente un gran numero di punti nel $\mathcal{C}$ -Space. Scarta quelli che cadono negli ostacoli ($\mathcal{O}$) e mantiene quelli in $\mathcal{C}_{free}$. Successivamente, tenta di connettere ogni nodo ai suoi k vicini più prossimi utilizzando un *Local Planner* semplice e veloce (come una linea retta interpolata). Se il segmento retto non interseca ostacoli, l'arco viene salvato. Il risultato è un grafo (la *Roadmap*) che cattura la topologia dello spazio libero.
2. **Fase di Query (Ricerca online):** Quando viene assegnato un task, il nodo di partenza q_{start} e il traguardo q_{goal} vengono temporaneamente agganciati ai nodi più vicini della Roadmap. A questo punto, il problema si riduce alla ricerca su un grafo spaziale, risolvibile istantaneamente con algoritmi standard come A^* o Dijkstra.

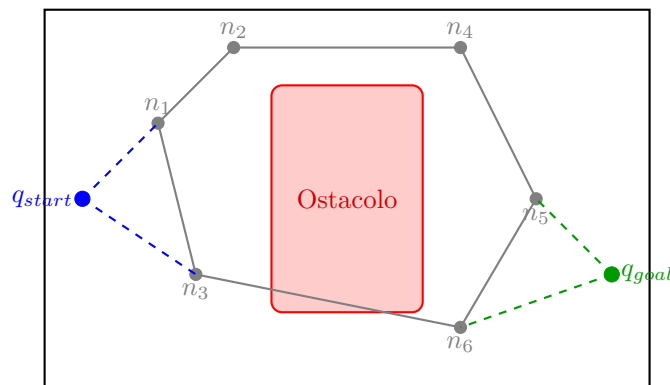


Figura 11: L'algoritmo PRM: la rete grigia è la Roadmap generata offline durante la fase di Learning. Le linee tratteggiate mostrano la fase di Query, in cui q_{start} e q_{goal} si agganciano al grafo per trovare un percorso.

7.2 Rapidly-exploring Random Trees (RRT)

A differenza di PRM, RRT è un algoritmo **single-query**: non costruisce una mappa globale per riutilizzarla, ma costruisce da zero una struttura ad albero specifica per risolvere il singolo problema corrente. È ideale per ambienti dinamici o ad altissima dimensionalità.

L'algoritmo fa crescere un albero radicato in q_{start} ripetendo iterativamente tre passaggi:

1. **Campionamento:** Genera un punto casuale q_{rand} nello spazio.
2. **Ricerca del vicino:** Trova il nodo $q_{nearest}$ attualmente nell'albero che è più vicino al punto q_{rand} .
3. **Estensione (Steering):** L'albero non compie un salto fino a q_{rand} , ma fa "un passo" in quella direzione (calcolo, non passo effettivo). Crea un nuovo nodo q_{new} avanzando da $q_{nearest}$ verso q_{rand} di una distanza fissa prestabilita Δq . Se il segmento tra $q_{nearest}$ e q_{new} non urta ostacoli, il nodo viene aggiunto all'albero.

Il segreto di RRT: Il Voronoi Bias

Ciò che rende RRT incredibilmente efficiente nell'esplorare rapidamente spazi complessi è il suo intrinseco **Voronoi Bias**. Se si divide lo spazio nelle Celle di Voronoi dell'albero (dove ogni cella rappresenta la zona di influenza di un nodo), i nodi di "frontiera" (quelli sui bordi esterni dell'albero) possiedono celle di area molto più vasta rispetto ai nodi intrappolati nel centro dell'albero.

Poiché q_{rand} viene campionato con probabilità uniforme in tutto lo spazio, è statisticamente molto più probabile che il campione cada nell'enorme cella di un nodo di frontiera. Questo "tira" costantemente la crescita dell'albero verso le aree non esplorate, impedendogli di concentrarsi inutilmente e raggrupparsi attorno al punto di partenza.

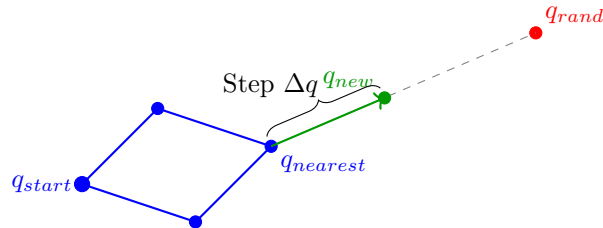


Figura 12: Espansione in RRT: campionato un punto random q_{rand} , si identifica il nodo più vicino $q_{nearest}$ e si estende l'albero di uno step fisso Δq per generare q_{new} .